LIBRARY MANAGEMENT SYSTEM

Python + MySQL | Beginner-Level Code Explanation and Viva Preparation

Purpose of this guide:

This guide explains the structure and logic of the alphabet-variable version of your Library Management System. It follows the complete Python program previously provided, which uses mysql.connector, datetime.date, database functions, and a 12-option menu.

Database assumed by the program: lib_mgmt_sys, with books, students, and issued_books tables.

1. Python import Statements

```
import mysql.connector

from datetime import date
```

import mysql.connector imports the MySQL Connector library. It allows Python to connect to a MySQL server and execute SQL statements. It does not itself connect to the database.

from datetime import date imports the date class from Python's built-in datetime module. The program uses date.today() to obtain the computer's current date when a book is issued or returned.

Viva: Why are these imports used? **Answer:** mysql.connector is used for communication between Python and MySQL. date is used to record the current issue date and return date.

2. Database connection Function:

```
def connect_database():
return mysql.connector.connect( host="localhost", user="root",
password="YOUR_MYSQL_PASSWORD", database="lib_mgmt_sys")
```

def defines a function. **connect_database** is the function name. The parentheses contain parameters; this function has none. The colon begins the function body. **return** sends the result back to the part of the program that called the function.

mysql.connector.connect() opens a connection to MySQL. host='localhost' means MySQL is running on the same computer. user='root' is the MySQL username. password must be replaced with the actual MySQL password. database='lib_mgmt_sys' selects the database.

The function is reused by other functions, so the connection details do not need to be written repeatedly.

3. General pattern used in database functions

```
a = connect_database() b = a.cursor()
b.execute("SQL QUERY") c = b.fetchall()
a.commit()
b.close()
a.close()
```

- **a** stores the database connection returned by connect_database().
- **b** stores a cursor. A cursor is an object used to send SQL commands to MySQL and read query results.
- **execute()** runs an SQL statement.
- **fetchone()** retrieves one result row. **fetchall()** retrieves all result rows.
- **commit()** saves changes made by INSERT, UPDATE, or DELETE.
- **close()** closes the cursor or database connection and releases resources.

Alphabet variables are short, but their meaning depends on the function. For example, a may mean a connection in one function and a menu choice in another. Read each function independently.

4. display_books()

```
def display_books(): try:
a = connect_database() b =
a.cursor()
b.execute("SELECT * FROM books") c = b.fetchall()
print("\n========== ALL BOOKS ==========") if c:
for d in c: print(d)
else:
print("No books available.")
b.close()
a.close() except mysql.connector.Error as e:
print("Database Error:", e)
```

This function displays every record in the books table.

- **try:** starts a block where an error may occur.
- **SELECT * FROM books** asks MySQL to return all columns and rows from books. The asterisk (*) means all columns.
- **fetchall()** stores all returned rows in c.
- **if c:** checks whether the result contains at least one row.
- **for d in c:** loops through the rows one by one. print(d) displays each row.
- **else:** runs when no book records are found.
- **except mysql.connector.Error as e:** catches MySQL-related errors and stores the error message in e.

5. add_book()

```
def add_book():
a = None b =
None try:
a = connect_database() b = a.cursor() c =
int(input("Enter Book ID: ")) d = input("Enter Book
Title: ") e = input("Enter Author Name: ") f =
input("Enter Category: ") g = float(input("Enter
Price: ")) h = int(input("Enter Quantity: ")) if g < 0
or h < 0:
print("Price and quantity cannot be negative.") return i = "INSERT INTO books
VALUES (%s, %s, %s, %s, %s, %s)"
```

```
j = (c, d, e, f, g, h)
b.execute(i, j)
a.commit() print("Book added successfully!")
except ValueError:
print("Invalid input. Enter valid numbers.") except mysql.connector.Error
as e:
print("Database Error:", e) finally: if b:
b.close() if a:
a.close()
```

The function takes book details from the user and inserts them into books.

- **a = None; b = None** gives the variables an initial value. This lets finally check whether a connection or cursor was successfully created.
- **input()** accepts text from the keyboard. **int()** converts input to an integer; **float()** converts it to a decimal number.
- **if g < 0 or h < 0** rejects a negative price or quantity. **or** means either condition being true is enough.
- **return** exits the function immediately.
- **%s** are parameter placeholders. The actual values are supplied separately in tuple j. This is parameterized SQL and is safer than joining user input directly into the query.
- **a.commit()** saves the inserted book.
- **ValueError** is caught if a numeric conversion such as int() or float() fails.
- **finally** runs whether the operation succeeds or an exception occurs. It closes resources if they were created.

6. search_book()

```
def search_book():
a = connect_database() b = a.cursor() c = input("Enter book
title to search: ")
b.execute( "SELECT * FROM books WHERE title LIKE %s",
("%" + c + "%",) ) d =
b.fetchall()
```

This function searches for books by title.

- **WHERE** filters records according to a condition.
- **LIKE** performs pattern matching on text.
- The percent sign (%) is a wildcard. '%python%' can match a title containing the word python anywhere.
- The comma in ("%" + c + "%",) makes it a one-item tuple, which is the format expected for the query parameters.

The remaining if/for/else logic displays matching rows or reports that no book was found. The cursor and connection are then closed. MySQL errors are caught by the except block.

7. add_student()

This function follows the same pattern as add_book(), but inserts student details into students.

```
c = int(input("Enter Student ID: ")) d = input("Enter
Student Name: ") e = input("Enter Class: ") f =
input("Enter Section: ") g = input("Enter Phone
Number: ")
h = "INSERT INTO students VALUES (%s, %s, %s, %s, %s)" i = (c, d, e, f, g)
```

```
b.execute(h, i)
a.commit()
```

The student ID is converted to an integer. Name, class, section, and phone are accepted as text. The values are inserted in the same order as the columns in the students table. commit() saves the new student.

8. display_students()

This function runs SELECT * FROM students, retrieves all rows with fetchall(), and prints each student record. If the result is empty, it prints 'No students registered.' It catches MySQL errors.

9. delete_student()

```
c = int(input("Enter Student ID to delete: "))
b.execute("SELECT * FROM students WHERE student_id = %s", (c,)) d = b.fetchone() if d is
None:
print("Student not found.") return
```

First, the program asks for a student ID and checks whether that student exists. fetchone() returns one matching row or None if there is no match.

```
e = input("Confirm deletion? (Y/N): ") if e.upper() != "Y":
print("Deletion cancelled.") return
```

upper() converts the response to uppercase, so both y and Y are accepted. If the answer is not Y, deletion is cancelled.

```
b.execute( "SELECT issue_id FROM issued_books "
"WHERE student_id = %s AND status = 'Issued'",
(c,) ) if b.fetchone():
print("Cannot delete student with an unreturned book.") return
```

This checks whether the student has any active issue record. If a matching row exists, fetchone() returns a row, which Python treats as true. The function refuses to delete a student who still has an unreturned book.

```
b.execute("DELETE FROM students WHERE student_id = %s", (c,))
a.commit()
```

DELETE removes the student record. Because the beginner-level table definition without CASCADE uses a foreign key, MySQL may reject deletion if any issue-history row still refers to that student. In that case, returned issue records must also be handled before deletion, or the database design must be changed.

10. issue_book()

```
c = int(input("Enter Book ID: ")) d = int(input("Enter
Student ID: "))
b.execute("SELECT quantity FROM books WHERE book_id = %s", (c,)) e = b.fetchone()
```

The function asks for the book ID and student ID. It checks the book's available quantity. If fetchone() returns None, the book ID does not exist.

```
b.execute("SELECT student_id FROM students WHERE student_id = %s", (d,)) f = b.fetchone() if f is None:
```

```
print("Student not found.") return if
e[0] <= 0:
print("Book is currently unavailable.") return
```

It checks that the student exists and that the book has at least one available copy. e[0] is the quantity value in the first column of the returned row.

```
b.execute( "INSERT INTO
issued_books "
"(book_id, student_id, issue_date, status) "
"VALUES (%s, %s, %s, %s)",
(c, d, date.today(), "Issued") )
b.execute( "UPDATE books SET quantity = quantity - 1 "
"WHERE book_id = %s",
(c,) )
a.commit()
```

The INSERT creates an issue record. The issue date is today's date and the status is Issued. The UPDATE reduces available book quantity by one. Both operations are saved with commit(). The code relies on issue_id being AUTO_INCREMENT in MySQL because it does not provide issue_id in the INSERT.

Important: If issue_id is a normal INT PRIMARY KEY without AUTO_INCREMENT, this INSERT will fail because no issue_id is supplied. In that design, the Python function must generate and insert an issue_id.

11. return_book()

```
c = int(input("Enter Book ID: ")) d = int(input("Enter
Student ID: "))
b.execute( "SELECT issue_id FROM issued_books "
"WHERE book_id = %s AND student_id = %s "
"AND status = 'Issued' ORDER BY issue_id",
(c, d) ) e = b.fetchone()
```

The function finds an active issue record for the given book and student. ORDER BY issue_id sorts matching records by issue ID. fetchone() selects the first matching record.

```
b.execute( "UPDATE
issued_books "
"SET return_date = %s, status = 'Returned' "
"WHERE issue_id = %s",
(date.today(), e[0]) )
b.execute( "UPDATE books SET quantity = quantity + 1 "
"WHERE book_id = %s",
(c,) )
a.commit()
```

The issue record is updated with today's return date and Returned status. The available quantity of the book is increased by one. commit() saves both updates.

12. update_book()

This function asks for a book ID, checks whether the book exists, then asks for the new title, author, category, price, and quantity.

```
b.execute( "UPDATE books SET title = %s, author = %s, "
"category = %s, price = %s, quantity = %s "
"WHERE book_id = %s",
(e, f, g, h, i, c) )
a.commit()
```

UPDATE changes existing data. SET lists the columns to change. WHERE ensures only the row with the entered book_id is changed. The values tuple must follow the same order as the placeholders.

The function checks that price and quantity are not negative before updating. In this project, quantity is treated as available copies because issuing decreases it and returning increases it. Avoid changing quantity arbitrarily while books are issued.

13. delete_book()

This function asks for a book ID, checks whether it exists, displays its details, and asks for confirmation. It then checks whether the book has an active issue record.

```
b.execute( "SELECT issue_id FROM issued_books "
"WHERE book_id = %s AND status = 'Issued'",
(c,) ) if b.fetchone():
print("Cannot delete a book that has not been returned.") return
```

If an active issue exists, deletion is stopped. Otherwise, DELETE FROM books removes the book and commit() saves the change. With foreign keys and no CASCADE, MySQL can still prevent deletion if old returned issue-history rows refer to the book. Those rows must be handled before deletion.

14. view_issued_books()

```
c = """ SELECT
issued_books.issue_id, books.title,
students.name,
issued_books.issue_date,
issued_books.return_date,
issued_books.status FROM
issued_books
INNER JOIN books
ON issued_books.book_id = books.book_id
INNER JOIN students ON issued_books.student_id =
students.student_id """
b.execute(c) d =
b.fetchall()
```

This function displays issue records together with the book title and student name.

• **Triple quotation marks** allow a multi-line SQL query to be stored in a Python string.

• **INNER JOIN** combines rows from related tables when the ON condition matches.

• The first join connects issued_books.book_id with books.book_id.

• The second join connects issued_books.student_id with students.student_id.

• The SELECT list chooses which columns to display.

The result is fetched with fetchall() and each row is printed. If no rows exist, the program displays 'No issue records found.'

15. Main menu

```
while True:
print("1. Display Books") print("2. Add Book")
... a = input("Enter your choice: ") if a == "1":
display_books() elif a == "2":
add_book() ... elif a
== "12":
print("Thank you for using the Library Management System.") break else:
print("Invalid choice. Please enter a number from 1 to 12.")
```

while True creates a loop that continues until break is executed. The menu is printed each time. input() reads the user's choice as text. if and elif select the matching function. break exits the loop when the user chooses 12. else handles an invalid choice.

16. Important Python concepts for viva

Concept	Meaning in this project
import	Loads a module or library.
def	Defines a function.
return	Sends a result back or exits a function.
try / except	Handles errors so the program does not stop unexpectedly.
finally	Runs cleanup code whether an error occurs or not.
if / elif / else	Makes decisions based on conditions.
while True	Repeats the menu until break.
for loop	Processes each row returned from MySQL.
input()	Accepts information from the user.
int() / float()	Convert input text into numeric values.
tuple	Groups values passed as SQL parameters.
None	Represents the absence of a value.
fetchone()	Gets one result row.
fetchall()	Gets all result rows.
commit()	Saves database changes.
rollback()	Cancels uncommitted changes after an error.

17. Important SQL concepts for viva

SQL concept	Meaning
CREATE TABLE	Creates a new table.
INSERT INTO	Adds a new record.
SQL concept	**Meaning**

SELECT	Retrieves records.
UPDATE	Changes existing records.
DELETE	Removes records.
WHERE	Filters records using a condition.
LIKE	Searches text using a pattern.
PRIMARY KEY	Uniquely identifies each row.
FOREIGN KEY	Creates a relationship between tables.
INNER JOIN	Combines matching records from tables.
VARCHAR	Stores variable-length text.
INT	Stores whole numbers.
DECIMAL	Stores decimal numbers.
DATE	Stores a date.
AUTO_INCREMENT	Automatically generates a numeric value; used by the original complete Python code's issue_id (

18. Likely viva questions and answers

Q. What is the aim of your project?

Ans. To manage book and student records and maintain book issue and return details using Python and MySQL.

Q. Why did you use MySQL?

Ans. MySQL stores the records in relational tables and allows the Python program to retrieve and modify them.

Q. Why did you use mysql.connector?

Ans. It provides the connection between Python and the MySQL database.

Q. Why did you use date.today()?

Ans. It obtains the computer's current date for issue and return records.

Q. What is a cursor?

Ans. A cursor is used to execute SQL statements and retrieve query results.

Q. What is a primary key?

Ans. A column or set of columns that uniquely identifies each row in a table.

Q. What is a foreign key?

Ans. A column that refers to a key in another table and establishes a relationship.

Q. Why is issue_id needed?

Ans. It uniquely identifies each borrowing transaction, including repeated borrowing of the same book.

Q. What is the difference between fetchone() and fetchall()?

Ans. fetchone() returns one row; fetchall() returns all remaining rows.

Q. Why is commit() used?

Ans. It permanently saves INSERT, UPDATE, and DELETE operations.

Q. What is INNER JOIN used for?

Ans. It displays related information from books, students, and issued_books in one result.

Q. What does LIKE do?

Ans. It searches text using a pattern, often with % as a wildcard.

Q. Why use parameterized queries?

Ans. They pass values separately from SQL text, helping prevent SQL injection and handling values correctly.

Q. What happens when a book is issued?

Ans. An issue record is inserted and the available quantity is decreased by one.

Q. What happens when a book is returned?

Ans. The issue record is marked Returned, the return date is stored, and available quantity increases by one.

Q. Why check quantity before issuing?

Ans. To prevent issuing a book when no copies are available.

Q. What is try-except?

Ans. It handles errors and lets the program show a message instead of terminating unexpectedly.

Q. What is the role of break?

Ans. It exits the menu loop when the user chooses Exit.

19. Suggested preparation order

First learn the purpose of the three tables and their primary/foreign keys. Next understand the connection function and the repeated cursor-execute-fetch-commit pattern. Then learn the four main operations: add, search/display, update/delete, and issue/return. Finally practise explaining the INNER JOIN in view_issued_books() and the menu loop.

Do not memorise answers without understanding the code. A viva examiner may change a value or ask what happens when a record is missing. Be prepared to explain the condition and the result.

END OF STUDY GUIDE